

Kickstarter Project Success Prediction

UE24CS352A Machine Learning Mini-Project • Problem #56 • Team: Aaditya Vijaywargi (PES1UG24AM003) & Amruta Sai (PES1UG24AM035)

1. Problem Statement

Given only information available when a Kickstarter campaign launches (funding goal, duration, category, location, launch date and the project blurb), predict whether the campaign will reach its funding goal (binary classification: success = 1, failure = 0). The project follows the approach of the CS229 report *Crowdfunding: Predicting Kickstarter Project Success*, which combines structured metadata with text features (TF-IDF, Naive Bayes and LDA topics). Post-launch variables such as amount pledged or backer count are deliberately excluded to avoid leakage.

2. Dataset Details

Important: the dataset is synthetic. *kickstarter_success_prediction_dataset.csv* (20,000 rows, 15 columns) was generated by *generate_dataset.py* (fixed seed 42) and is not scraped from Kickstarter. Its patterns are modelled on widely reported Kickstarter trends: larger goals fail more often, very short or very long campaigns do worse, category success rates differ (Dance, Theater and Comics high; Technology, Journalism and Food low), and specific, prepared-sounding blurbs correlate with success while vague hype correlates with failure. A hidden project-quality variable influences both the blurb wording and the outcome, plus random noise, so no model can be near-perfect. Because the data is synthetic, absolute accuracy should not be compared with results on real Kickstarter data.

Attribute	Details
Rows / target	20,000 projects; outcome 1 = success (54.8%), 0 = failure (45.2%)
Numeric	name_length, blurb_length, goal_usd, duration_days, launch year / month / weekday (derived)
Categorical	currency, country, parent_category (15), category (50), location, location_type
Text	blurb (about 19,000 distinct blurbs)
Split	Stratified 70/30 train/test (14,000 / 6,000), random_state = 42

3. Approach

- **Metadata pipeline:** median imputation and standard scaling for numeric columns; one-hot encoding for categorical columns.
- **Text model:** blurbs are cleaned and vectorised with TF-IDF; a Multinomial Naive Bayes model outputs $P(\text{success} | \text{text})$, used as a stacked feature. For the training set this probability is produced with 5-fold *out-of-fold* predictions, so each row is scored by a model that never saw its label (avoids leakage).
- **Topic features:** LDA (10 topics) on bag-of-words counts adds a topic distribution per project.
- **Models compared:** Logistic Regression, L1 Logistic (Lasso-style), Ridge, Linear SVM, Random Forest, Neural Network (MLP) and Gradient Boosting, all trained on the same 174 features (metadata + Naive Bayes probability + 10 LDA topics). Gradient Boosting was then tuned with 5-fold GridSearchCV (24 candidates, 120 fits).

4. Implementation Overview

Everything is in one notebook, *Kickstarter_Success_Prediction.ipynb*, run top to bottom: load data, inspect and clean, create launch-time features, EDA, split, preprocess metadata, train the text model, add LDA features, compare models, evaluate (confusion matrix, ROC curves, feature importance), tune, save artifacts, and expose a *predict_project()* function that takes a new project's fields and returns a prediction with probabilities. Saved outputs: *models/*.pkl* (preprocessor, TF-IDF, Naive Bayes, count vectorizer, LDA, Gradient Boosting), *model_results.csv* and *test_predictions.csv*. Dependencies are listed in *requirements.txt*; the README explains setup.

5. Results (held-out test set, 6,000 projects)

A classifier that always predicts "success" scores 54.8% accuracy (ROC-AUC 0.50), so that is the baseline to beat.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Gradient Boosting	0.715	0.732	0.757	0.744	0.790
Random Forest	0.696	0.731	0.703	0.717	0.771
Neural Network	0.691	0.703	0.755	0.728	0.763
L1 Logistic (Lasso-style)	0.691	0.715	0.726	0.720	0.759
Logistic Regression	0.686	0.710	0.723	0.716	0.758
Linear SVM	0.683	0.709	0.716	0.712	0.749
Ridge	0.645	0.685	0.651	0.668	0.701
Majority baseline	0.548	–	–	–	0.500

- **Gradient Boosting is best** (71.5% accuracy, AUC 0.790), about 16.7 points above the baseline. Tree ensembles beat linear models, suggesting nonlinear effects such as goal size relative to category.
- **Tuning:** best parameters were `learning_rate 0.1`, `max_depth 3`, `max_features 0.5`, `n_estimators 100` (CV accuracy 0.714; test accuracy 0.712, F1 0.742). Tuning did not improve on the default model, which is plausible because signal in the data is moderate.
- **Balanced errors:** failed-project recall is 0.66 and successful-project recall is 0.76, so the model does not simply predict the majority class.
- **Feature importance:** `goal_usd` is dominant (about 45%), followed by the Naive Bayes text probability (about 17%); `duration`, category indicators and LDA topics contribute smaller amounts. Text alone (Naive Bayes) reaches 62.1% accuracy, showing the blurb carries real but limited signal.

6. Conclusions and Limitations

Combining metadata, a Naive Bayes text score and LDA topics in a Gradient Boosting classifier gave the best result (71.5% accuracy, AUC 0.79 against a 54.8% baseline), confirming that funding goal and blurb content are the most informative launch-time signals. Main limitations: (1) the data is synthetic, so conclusions describe the generating process rather than real crowdfunding behaviour; the same pipeline should be rerun on real Kickstarter data (for example the Web Robots scrape) to measure real performance; (2) LDA topics add little on this data because blurbs are built from a limited vocabulary; (3) only launch-time features are used, so updates, images and video are not captured. Future work: real data, word embeddings or transformer text features, and probability calibration.